GrokRxiv:2026.03.memory-layer-ai-os [cs.AI] 10 Mar 2026

The AI Operating System, Part III: Memory Layer — Typed Filesystem for Persistent Agent Cognition

Matthew Long
The YonedaAI Collaboration
YonedaAI Research Collective
Chicago, IL
matthew@yonedaai.com · https://yonedaai.com

March 10, 2026

Abstract

Autonomous AI agents require persistent memory that transcends session boundaries, yet no analogue of the operating-system filesystem has been designed specifically for agent cognition. Traditional filesystems store untyped byte streams; agent memory demands *typed schemas*, *causal versioning*, *graph-structured lineage*, and *multi-backend storage coordination*. We present the design and implementation of a *typed memory layer* for the AI Operating System: a system built on Elixir/OTP that provides 24 distinct memory types organized into six subcategories, a schema registry for runtime type resolution, content-addressed versioning with causal tracking via vector clocks, a graph relationship manager for memory lineage, and a multi-backend storage router supporting ETS for working memory and Mnesia for persistent knowledge. Each memory instance is a GenServer process holding typed state, enabling per-memory isolation, concurrent access, and fault-tolerant supervision. The design draws on functional programming principles—immutable data structures, pattern matching, behaviour protocols, and pipeline composition—to achieve composability and type safety without sacrificing pragmatism. The reference implementation, ContextFS, realizes this framework in production with an MCP server for universal client access. We provide the complete Elixir source, demonstrate the architecture through three case studies (agent learning, cross-session continuity, and multi-agent collaboration), and discuss how the design relates to classical OS memory management, knowledge graphs, and cognitive architectures.

Keywords: agent memory, typed filesystem, Elixir/OTP, knowledge representation, persistent cognition, GenServer, ETS, Mnesia, MCP

Contents

1 Introduction **4**
1.1 Paper Organization 5

2 Background **5**
2.1 OS Memory Management and Filesystems 5
2.2 Knowledge Representation 5
2.3 Embedding Spaces and Semantic Search 6
2.4 Elixir/OTP and the BEAM Virtual Machine 6
2.5 Design Inspiration: Category Theory 6

3 Design Overview **7**

4 Type System and Schema Registry **8**
4.1 Design Rationale . . . 8
4.2 BaseSchema: The Universal Type . . . 9
4.3 The 24 Memory Types . . . 9
4.4 Domain Schema Definitions . . . 11
4.5 The Schema Registry . . . 13
4.6 Type Safety: Two Levels . . . 15

5 The Memory GenServer **15**
5.1 Process-Per-Memory Architecture . . . 15
5.2 Creation and Initialization . . . 16
5.3 Content Hashing . . . 17
5.4 Updates with Deduplication . . . 17
5.5 Evolution: Parent-Child Lineage . . . 17
5.6 Merge: Conflict Resolution . . . 18
5.7 Soft Deletes . . . 20
5.8 Persistence: Dual-Layer Write Path . . . 20

6 Versioning and Causal Tracking **21**
6.1 Change Reasons . . . 21
6.2 Version Entries . . . 22
6.3 History and Lineage Traversal . . . 22
6.4 Vector Clocks . . . 23

7 Graph Relationships **24**
7.1 Edge Relations . . . 24
7.2 Link and Unlink . . . 25
7.3 Graph Traversal . . . 26
7.4 Neighborhood Queries . . . 27
7.5 Conflict Detection . . . 27

8 Multi-Backend Storage Router **28**
8.1 The StorageBackend Behaviour . . . 28
8.2 The Storage Router . . . 28
8.3 Tiered Recall with Automatic Promotion . . . 29
8.4 Cross-Backend Search . . . 30
8.5 Text Matching and Filtering . . . 30
8.6 LRU Eviction . . . 31
8.7 Working Memory vs. Persistent Knowledge . . . 32

9 Reliability Guarantees **32**
9.1 ACID Properties via Mnesia Transactions . . . 32
9.2 Type Preservation Under Round-Trip . . . 32
9.3 Deduplication Soundness . . . 33
9.4 Conflict Detection Completeness . . . 33
9.5 Soft Delete Lineage Preservation . . . 33

10 MCP Server Interface **33**

11 Case Studies **34**
11.1 Agent Learning from Past Executions 34
11.2 Cross-Session Continuity 35
11.3 Multi-Agent Shared Knowledge Base 36

12 Discussion **37**
12.1 Comparison to Unix VFS 37
12.2 Comparison to Plan 9 37
12.3 Comparison to Knowledge Graphs 37
12.4 Relation to Cognitive Architectures 37
12.5 Connections to Category Theory 38
12.6 Relation to Version Control 38
12.7 Limitations and Future Work 39

13 Related Work **39**

14 Conclusion **40**

A Complete Memory Type Hierarchy **42**

B OTP Application Startup **43**

C Mix Project Configuration **44**

D Version Chain Diagram **44**

1 Introduction

The first two installments of this series established the kernel architecture and the scheduler for an AI Operating System. Yet an operating system without a memory subsystem is merely a stateless dispatcher: it can schedule computations but cannot accumulate knowledge across them. Every traditional OS dedicates a substantial fraction of its complexity to memory management—virtual memory, page tables, filesystem hierarchies, caching layers—because persistent, organized storage is the foundation upon which all higher-level abstractions rest.

For autonomous AI agents, the memory problem is simultaneously more constrained and more ambitious than its classical counterpart. More constrained because agents do not need byte-addressable random access to flat address spaces; more ambitious because agent memory must support:

(i) **Typed schemas**: each memory entry conforms to a domain-specific type, not a raw byte stream.

(ii) **Causal versioning**: memory evolves over time, and the system must track *why* each change occurred (observation, inference, correction, decay).

(iii) **Graph lineage**: memories relate to one another through typed edges—one memory may supersede, contradict, reference, or evolve into another.

(iv) **Multi-backend coordination**: different storage engines excel at different query patterns, and the memory layer must route transparently.

(v) **Concurrent access**: multiple agents and sessions must read and write shared memories without corruption.

Traditional filesystems fail on every count. POSIX provides `open/read/write/close` on untyped byte sequences organized in a tree hierarchy. Plan 9's union mounts and synthetic filesystems [Pike et al., 1995] introduced compositional namespace ideas but remained fundamentally untyped. Semantic filesystems [Gifford et al., 1991] added metadata-based retrieval but lacked formal type guarantees. None of these systems address versioning with causal annotations, graph-structured relationships, or the concurrent multi-agent access patterns that agent cognition requires.

This paper presents the *design and implementation* of a typed memory layer for agent cognition, realized in Elixir/OTP and deployed as ContextFS. Rather than leading with mathematical formalism, we lead with the engineering: the data structures, the protocols, the process architecture, and the code. Where the underlying ideas have roots in type theory or category theory, we note the connection, but the Elixir implementation is the primary artifact.

The design yields three concrete benefits:

1. **Composability**: memory operations compose through Elixir's pipeline operator (`|>`), behaviour protocols, and GenServer message passing, enabling complex workflows from simple building blocks.

2. **Type safety**: a combination of Dialyzer typespecs (compile-time) and struct enforcement with `@enforce_keys` (runtime) prevents type confusion across the 24 memory types.

3. **Extensibility**: new schema types, storage backends, and relationship kinds can be added by implementing existing behaviours and registering with the schema registry—no modification to core modules required.

1.1 Paper Organization

Section 2 surveys relevant background in OS memory management, knowledge representation, and the Elixir/OTP platform. Section 3 presents the high-level architecture. Section 4 details the type system and schema registry. Section 5 presents the core Memory GenServer. Section 6 describes versioning with causal tracking and vector clocks. Section 7 covers the graph relationship manager. Section 8 presents the multi-backend storage router. Section 9 establishes reliability guarantees. Section 10 describes the MCP server interface. Section 11 validates the framework through case studies. Section 12 discusses related work and connections to formal frameworks. Section 14 concludes.

2 Background

2.1 OS Memory Management and Filesystems

The virtual memory subsystem of a modern OS provides each process with an isolated address space mapped onto physical memory and secondary storage through page tables [Denning, 1970]. The key abstraction is *indirection*: processes reference virtual addresses, and the memory management unit translates these to physical locations. This indirection enables demand paging, copy-on-write, and shared memory regions.

Filesystems layer a persistent namespace on top of block devices. The Unix filesystem [Ritchie and Thompson, 1974] introduced the inode/directory model: files are sequences of bytes identified by inodes, organized in a tree of directories. The Virtual Filesystem Switch (VFS) in Linux [Love, 2010] provides a uniform interface over heterogeneous implementations—ext4, XFS, NFS, FUSE—through a common set of operations (`inode_operations`, `file_operations`). Plan 9 [Pike et al., 1995] radicalized the "everything is a file" philosophy, presenting network connections, process state, and device control as synthetic file servers.

These designs are profoundly influential, yet they share a fundamental limitation: *files are untyped byte streams*. The filesystem knows nothing about the structure of what it stores. All interpretation is deferred to applications.

2.2 Knowledge Representation

The knowledge representation community has long studied how to organize structured information for intelligent systems. Frame systems [Minsky, 1974] organize knowledge around stereotypical situations with slots and defaults. Semantic networks [Quillian, 1968] represent concepts as nodes and relationships as labeled edges. Knowledge graphs [Hogan et al., 2021] have become the dominant paradigm, exemplified by Wikidata and domain-specific graphs in biomedicine and finance.

The connection to agent memory is direct: an agent's accumulated knowledge forms a personal knowledge graph, with the crucial addition that this graph *evolves over time* as the agent observes, infers, corrects, and forgets.

2.3 Embedding Spaces and Semantic Search

Transformer-based language models produce dense vector representations that capture semantic similarity [Reimers and Gurevych, 2019, Devlin et al., 2019]. Retrieval-augmented generation (RAG) [Lewis et al., 2020] exploits this by storing text chunks as vectors and retrieving relevant context at query time through approximate nearest neighbor search. For agent memory, embeddings provide a capability that neither filesystems nor knowledge graphs offer natively: *retrieval by meaning*.

2.4 Elixir/OTP and the beam Virtual Machine

Elixir is a functional programming language that runs on the BEAM virtual machine [Armstrong, 2013, Thomas and Hunt, 2018]. The BEAM was designed for building fault-tolerant, distributed, soft-real-time systems. Several BEAM features make it uniquely suited for implementing an agent memory layer:

- **Lightweight processes**: the BEAM supports millions of concurrent processes, each with isolated memory. A per-memory GenServer process provides natural isolation and concurrent access.
- **OTP behaviours**: GenServer, Supervisor, and Application provide battle-tested patterns for stateful services, fault recovery, and lifecycle management.
- **ETS (Erlang Term Storage)**: in-memory key-value tables with concurrent read/write access and constant-time lookups. Ideal for working memory.
- **Mnesia**: a distributed, real-time database built into OTP. Supports transactions, replication, schema migration, and both RAM and disk storage. Ideal for persistent knowledge.
- **Pattern matching**: Elixir's pervasive pattern matching enables concise, type-safe dispatch across the 24 memory types.
- **Immutable data**: all Elixir data structures are immutable by default, eliminating shared-state concurrency bugs.
- **Pipeline composition**: the `|>` operator enables readable data transformation chains.
- **Typespecs and Dialyzer**: optional static type checking via `@type`, `@spec`, and the Dialyzer tool catches type mismatches at compile time.

2.5 Design Inspiration: Category Theory

The design of the memory layer is informed by ideas from category theory, though we do not develop a full categorical formalism in this paper. The key insights borrowed from this tradition are:

- **Typed mappings preserve structure**: the memory system maps typed schemas to storage representations in a way that preserves type information across round-trips. This is analogous to a functor preserving structure between categories.

- **Operations compose**: schema transformations, storage queries, and graph traversals compose associatively, just as morphisms compose in a category.
- **Universal constructions**: the BaseSchema serves as a universal type from which all 24 schema types specialize, analogous to an initial object. The storage router aggregates backend capabilities, analogous to a coproduct.

Section 12 elaborates on these connections for readers interested in the formal underpinnings.

3 Design Overview

The memory layer is an OTP application consisting of five core modules organized in a supervision tree. Figure 1 shows the high-level architecture.

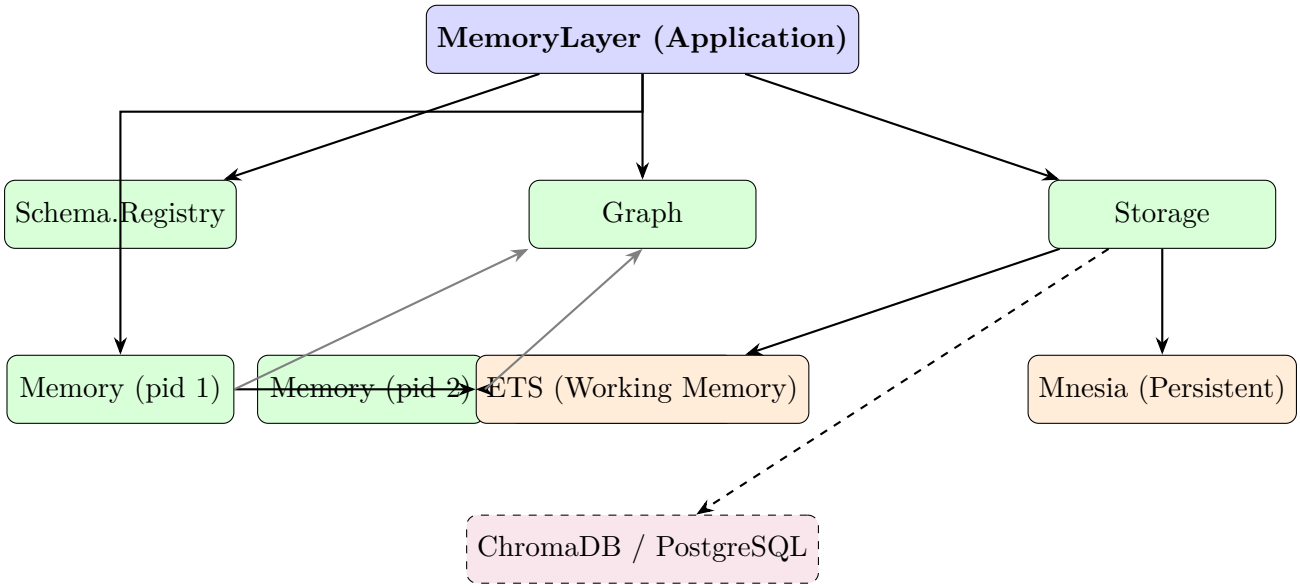


Figure 1: Memory Layer architecture. Solid lines show OTP supervision and data flow; dashed lines show planned extensions. Each Memory instance is a separate GenServer process.

Design Principle 1 (Everything an agent knows is a typed memory)**.** Just as Plan 9 made everything a file, the memory layer makes everything a typed memory instance—facts, decisions, procedures, episodes, tasks, configurations, and even the agent's own execution traces.

Design Principle 2 (Dual-layer storage mirrors cognition)**.** ETS serves as working memory (fast, volatile, bounded); Mnesia serves as long-term memory (durable, queryable, unbounded). Promotion and demotion between layers happens automatically based on access patterns.

Design Principle 3 (Per-memory process isolation)**.** Each memory instance runs as its own GenServer process, providing natural concurrency isolation, independent lifecycle management, and the ability to supervise individual memories.

The five core modules and their responsibilities:

1. **`MemoryLayer.Schema`** — Type system: 24 memory type definitions as Elixir structs, subcategory classification, and the `Registry` GenServer for runtime type resolution.
2. **`MemoryLayer.Memory`** — Core GenServer: per-memory stateful process with typed data, content hashing, and operations (create, wrap, data, update, evolve, merge, delete).
3. **`MemoryLayer.Version`** — Versioning: causal change tracking with parent-child lineage, change reason annotations, and vector clocks for multi-agent conflict detection.
4. **`MemoryLayer.Graph`** — Relationships: nine typed edge relations between memories, graph traversal (BFS with depth limits), neighborhood queries, and conflict detection.
5. **`MemoryLayer.Storage`** — Multi-backend router: fan-out writes to ETS and Mnesia, tiered recall with automatic promotion, cross-backend search, and LRU eviction.

Table 1 summarizes the OTP process types:

Table 1: OTP process types in the memory layer.

Module	Process Type	State
`MemoryLayer`	Application	Initializes ETS tables, starts Mnesia, launches supervisor
`Schema.Registry`	Named GenServer	Map from schema name strings to module atoms
`Memory`	Per-instance GenServer	Typed data struct, version, content hash, vector clock, timestamps
`Graph`	Named GenServer	Stateless coordinator (edges stored in Mnesia)
`Storage`	Named GenServer	Stateless coordinator (data in ETS + Mnesia)

4 Type System and Schema Registry

4.1 Design Rationale

The fundamental design decision in the memory layer is that *every memory has a type*. Unlike traditional filesystems where a file is an opaque byte sequence, each memory instance carries a schema that defines its fields, their types, and their constraints. This enables:

- Compile-time checking via Dialyzer typespecs
- Runtime validation via `@enforce_keys` and struct construction
- Type-safe deserialization: given a stored `schema_name` string, the registry resolves the correct Elixir module for reconstruction
- Domain-specific operations: a `TodoData` memory can be queried by priority and status; a `FactData` memory can be filtered by confidence

4.2 BaseSchema: The Universal Type

All 24 memory types share a common envelope defined by `BaseSchema`:

```
defmodule MemoryLayer.Schema.BaseSchema do
  @enforce_keys [:schema_name, :content]
  defstruct [
    :schema_name,
    :content,
    :metadata,
    id: nil,
    created_at: nil,
    updated_at: nil,
    deleted_at: nil,
    content_hash: nil,
    tags: []
  ]

  @type t :: %__MODULE__{
    schema_name: String.t(),
    content: map(),
    metadata: map() | nil,
    id: String.t() | nil,
    created_at: DateTime.t() | nil,
    updated_at: DateTime.t() | nil,
    deleted_at: DateTime.t() | nil,
    content_hash: String.t() | nil,
    tags: [String.t()]
  }
end
```

Listing 1: BaseSchema: the universal memory envelope.

The `@enforce_keys` directive ensures that `schema_name` and `content` must be provided at construction time—the Elixir compiler will raise an error if they are omitted. The `deleted_at` field supports soft deletes (Section 5.7), and `content_hash` enables deduplication (Section 5.3).

4.3 The 24 Memory Types

ContextFS defines 24 memory types organized into six subcategories. The classification is defined by pattern matching on the type atom:

```
defmodule MemoryLayer.Schema do
  @type memory_type ::
    :fact | :decision | :procedural | :episodic |
    :todo | :issue | :api | :schema_def |
    :workflow | :task | :step | :agent_run |
    :tag | :annotation | :embedding | :index |
    :config | :session | :context | :log |
    :person | :project | :artifact | :event

  @type subcategory ::
```

```
    :core | :extended | :workflow |
    :metadata | :system | :relational

  def subcategory(t) when t in [:fact, :decision,
    :procedural, :episodic], do: :core
  def subcategory(t) when t in [:todo, :issue,
    :api, :schema_def], do: :extended
  def subcategory(t) when t in [:workflow, :task,
    :step, :agent_run], do: :workflow
  def subcategory(t) when t in [:tag, :annotation,
    :embedding, :index], do: :metadata
  def subcategory(t) when t in [:config, :session,
    :context, :log], do: :system
  def subcategory(t) when t in [:person, :project,
    :artifact, :event], do: :relational
end
```

Listing 2: Subcategory classification via pattern matching.

Table 2 lists all 24 types with their fields and intended use:

Table 2: The 24 memory types in ContextFS.

Subcategory	Type	Purpose and Key Fields
Core	`fact`	Declarative knowledge: `assertion`, `confidence`, `evidence`, `source`
	`decision`	Decision records: `decision`, `rationale`, `alternatives`, `confidence`
	`procedural`	How-to knowledge: `name`, `steps`, `preconditions`, `postconditions`
	`episodic`	Experiential records: `event`, `outcome`, `context`, `duration_ms`
Extended	`todo`	Action items: `title`, `priority` (low/med/high/critical), `status`
	`issue`	Problem records: `title`, `severity`, `root_cause`, `resolution`
	`api`	API schemas and access patterns
	`schema_def`	Meta-level: schemas about schemas
Workflow	`workflow`	End-to-end processes: `name`, `steps`, `trigger`, `status`
	`task`	Individual work units within workflows
	`step`	Atomic operations within tasks
	`agent_run`	Execution traces: `agent_id`, `task`, `model`, `duration_ms`, `status`
Metadata	`tag`	Classification labels
	`annotation`	Commentary on other memories
	`embedding`	Vector representations
	`index`	Search index entries
System	`config`	Configuration parameters
	`session`	Agent session state
	`context`	Contextual information bundles
	`log`	System event records
Relational	`person`	People known to the agent
	`project`	Project-level organizational units
	`artifact`	Produced outputs (documents, code, analyses)
	`event`	Calendar or timeline events

4.4 Domain Schema Definitions

Each memory type is an Elixir struct with enforced keys and typespecs. Here are three representative examples spanning the core, extended, and workflow subcategories:

```elixir
defmodule MemoryLayer.Schema.FactData do
  @moduledoc "Verified assertions with confidence."

  @enforce_keys [:assertion]
  defstruct [
    :assertion,
```

```
    :evidence,
    :source,
    confidence: 1.0,
    schema_name: "fact"
  ]

  @type t :: %__MODULE__{
    assertion: String.t(),
    evidence: String.t() | nil,
    source: String.t() | nil,
    confidence: float(),
    schema_name: String.t()
  }
end
```

Listing 3: FactData: declarative knowledge with confidence tracking.

```
defmodule MemoryLayer.Schema.IssueData do
  @enforce_keys [:title, :severity]
  defstruct [
    :title,
    :severity,
    :description,
    :resolution,
    :root_cause,
    status: :open,
    schema_name: "issue"
  ]

  @type t :: %__MODULE__{
    title: String.t(),
    severity: :low | :medium | :high | :critical,
    description: String.t() | nil,
    resolution: String.t() | nil,
    root_cause: String.t() | nil,
    status: :open | :investigating | :resolved | :closed,
    schema_name: String.t()
  }
end
```

Listing 4: IssueData: problem tracking with severity and resolution.

```
defmodule MemoryLayer.Schema.AgentRunData do
  @enforce_keys [:agent_id, :task]
  defstruct [
    :agent_id, :task, :result, :error,
    :input, :output, :duration_ms, :model,
    status: :running,
    started_at: nil,
    completed_at: nil,
    schema_name: "agent_run"
  ]
```

```

  @type t :: %__MODULE__{
    agent_id: String.t(),
    task: String.t(),
    result: term() | nil,
    error: String.t() | nil,
    input: map() | nil,
    output: map() | nil,
    duration_ms: non_neg_integer() | nil,
    model: String.t() | nil,
    status: :running | :completed | :failed | :cancelled,
    started_at: DateTime.t() | nil,
    completed_at: DateTime.t() | nil,
    schema_name: String.t()
  }
end
```

Listing 5: AgentRunData: execution trace of an agent invocation.

The pattern is consistent across all 24 types: `@enforce_keys` declares the mandatory fields, `defstruct` provides defaults for optional fields, and `@type` gives the complete typespec for Dialyzer. The `schema_name` field is always defaulted to the type's string identifier, linking the struct to its registry entry.

4.5 The Schema Registry

The `Schema.Registry` is a GenServer that maps schema name strings to their Elixir modules, enabling type-safe deserialization at runtime:

```
defmodule MemoryLayer.Schema.Registry do
  use GenServer

  @type registry_state :: %{String.t() => module()}

  def start_link(opts \\ []) do
    GenServer.start_link(__MODULE__, opts, name: __MODULE__)
  end

  @spec resolve(String.t()) ::
    {:ok, module()} | {:error, :not_found}
  def resolve(schema_name) do
    GenServer.call(__MODULE__, {:resolve, schema_name})
  end

  @spec register(String.t(), module()) :: :ok
  def register(schema_name, module) do
    GenServer.call(__MODULE__,
      {:register, schema_name, module})
  end

  @spec list() :: [String.t()]
  def list do
```

```
    GenServer.call(__MODULE__, :list)
  end

  # --- GenServer Callbacks ---

  @impl true
  def init(_opts) do
    registry = %{
      "fact"       => MemoryLayer.Schema.FactData,
      "decision"   => MemoryLayer.Schema.DecisionData,
      "procedural" => MemoryLayer.Schema.ProceduralData,
      "episodic"   => MemoryLayer.Schema.EpisodicData,
      "todo"       => MemoryLayer.Schema.TodoData,
      "issue"      => MemoryLayer.Schema.IssueData,
      "workflow"   => MemoryLayer.Schema.WorkflowData,
      "agent_run"  => MemoryLayer.Schema.AgentRunData
    }
    {:ok, registry}
  end

  @impl true
  def handle_call({:resolve, name}, _from, state) do
    case Map.fetch(state, name) do
      {:ok, module} -> {:reply, {:ok, module}, state}
      :error -> {:reply, {:error, :not_found}, state}
    end
  end

  @impl true
  def handle_call({:register, name, module}, _from, state) do
    {:reply, :ok, Map.put(state, name, module)}
  end

  @impl true
  def handle_call(:list, _from, state) do
    {:reply, Map.keys(state), state}
  end
end
```

Listing 6: Schema Registry: runtime type resolution.

The registry is initialized with the built-in types at application startup, and new types can be registered dynamically via `register/2`. The mapping is injective by construction: each schema name maps to exactly one module, ensuring unambiguous type resolution.

Remark 4.1 (Extensibility)**.** Third-party schema types can be registered at runtime without modifying the core `Schema` module. An agent that discovers a new domain can define a struct, register it, and immediately begin creating typed memories of that new type.

4.6 Type Safety: Two Levels

Type safety in the memory layer operates at two complementary levels:

Compile-time (Dialyzer). Every public function carries an `@spec` annotation. The Dialyzer static analysis tool traces type information through the codebase and reports mismatches. For example, if a caller passes a `FactData` struct to a function expecting `IssueData`, Dialyzer will flag this at compile time.

Runtime (struct enforcement). Elixir's `@enforce_keys` and `defstruct` validate data at construction time. Attempting to create a `FactData` without the required `:assertion` field raises an `ArgumentError` immediately, preventing invalid memories from entering the system. The `is_struct/1` guard in the Memory GenServer's `create/1` function further ensures only valid structs are accepted:

```
@spec create(struct()) :: {:ok, pid()} | {:error, term()}
def create(schema_data) when is_struct(schema_data) do
  GenServer.start_link(__MODULE__, {:create, schema_data})
end
```

Listing 7: Guard clause ensures only structs enter the system.

5 The Memory GenServer

5.1 Process-Per-Memory Architecture

The central design choice in the memory layer is that each memory instance is a GenServer process. This is a natural fit for the BEAM: lightweight processes are cheap (a few hundred bytes of initial memory), and the BEAM scheduler handles millions of them efficiently.

Each Memory process holds the complete typed state for one memory instance:

```
defmodule MemoryLayer.Memory do
  use GenServer

  alias MemoryLayer.{Version, Storage, Graph}

  @type t :: %{
    id: String.t(),
    schema_name: String.t(),
    data: struct(),
    version: non_neg_integer(),
    content_hash: String.t(),
    vector_clock: map(),
    created_at: DateTime.t(),
    updated_at: DateTime.t(),
    deleted_at: DateTime.t() | nil
  }
```

Listing 8: Memory GenServer state type.

The `data` field holds the typed schema struct (e.g., a `FactData` or `IssueData`). The `vector_clock` tracks causal ordering across agents (Section 6). The `content_hash` enables deduplication (Section 5.3).

5.2 Creation and Initialization

Creating a new memory starts a GenServer process, computes a UUID and content hash, and persists to both storage tiers:

```
@spec create(struct()) :: {:ok, pid()} | {:error, term()}
def create(schema_data) when is_struct(schema_data) do
  GenServer.start_link(__MODULE__, {:create, schema_data})
end

@impl true
def init({:create, schema_data}) do
  case build_state(schema_data) do
    {:ok, state} ->
      persist(state)
      {:ok, state}
    {:error, reason} ->
      {:stop, reason}
  end
end

defp build_state(schema_data, opts \\ []) do
  id = Keyword.get(opts, :id) || generate_uuid()
  now = DateTime.utc_now()

  {:ok, %{
    id: id,
    schema_name: extract_schema_name(schema_data),
    data: schema_data,
    version: 1,
    content_hash: content_hash(schema_data),
    vector_clock: %{node() => 1},
    created_at: now,
    updated_at: now,
    deleted_at: nil
  }}
rescue
  e -> {:error, Exception.message(e)}
end
```

Listing 9: Memory creation: from schema struct to supervised process.

Several details are worth noting:

- The `vector_clock` is initialized with the current BEAM node as the sole entry, set to 1. In a distributed deployment, each node contributes its own clock component.
- The `rescue` clause catches struct validation errors (e.g., missing enforced keys) and converts them to `{:error, reason}` tuples, preventing the GenServer from crashing on invalid input.
- The `extract_schema_name/1` function extracts the type tag using either the struct's `schema_name` field or, as a fallback, the module name.

5.3 Content Hashing

Every memory instance receives a SHA-256 content hash computed from its data:

```
@spec content_hash(struct()) :: String.t()
def content_hash(data) do
  data
  |> :erlang.term_to_binary()
  |> then(&:crypto.hash(:sha256, &1))
  |> Base.encode16(case: :lower)
end
```

Listing 10: Content-addressed hashing for deduplication.

This three-step pipeline—serialize, hash, encode—produces a deterministic fingerprint for any Elixir term. Two memories with identical typed data will have the same hash regardless of when or by whom they were created. The `update/2` handler uses this for deduplication: if the hash has not changed after applying updates, the write is skipped entirely.

5.4 Updates with Deduplication

The update handler applies changes to the underlying struct, recomputes the hash, and short-circuits if the content is unchanged:

```
@impl true
def handle_call({:update, changes}, _from, state) do
  updated_data = struct(state.data, Map.to_list(changes))
  new_hash = content_hash(updated_data)

  if new_hash == state.content_hash do
    {:reply, :ok, state}  # No change, skip write
  else
    new_state = %{state |
      data: updated_data,
      version: state.version + 1,
      content_hash: new_hash,
      updated_at: DateTime.utc_now()
    }
    persist(new_state)
    {:reply, :ok, new_state}
  end
end
```

Listing 11: Update with content-hash deduplication.

The `struct/2` function applies a keyword list of changes to an existing struct, raising if any key does not exist in the struct definition. This provides an additional layer of runtime type safety: attempting to set a nonexistent field will fail immediately.

5.5 Evolution: Parent-Child Lineage

The `evolve/3` operation creates a new memory (child) from an existing one (parent), establishing a causal link:

```elixir
@spec evolve(pid(), map(), Version.change_reason()) ::
  {:ok, pid()} | {:error, term()}
def evolve(pid, changes, reason) do
  GenServer.call(pid, {:evolve, changes, reason})
end

@impl true
def handle_call({:evolve, changes, reason}, _from, state) do
  evolved_data = struct(state.data, Map.to_list(changes))

  case create(evolved_data) do
    {:ok, child_pid} ->
      {:ok, child_state} =
        GenServer.call(child_pid, :get_state)

      # Record version lineage in Mnesia
      Version.record(
        state.id, child_state.id,
        reason, state.vector_clock
      )

      # Establish graph edge
      Graph.link(
        state.id, child_state.id,
        :evolved_into, %{reason: reason}
      )

      {:reply, {:ok, child_pid}, state}

    {:error, reason} ->
      {:reply, {:error, reason}, state}
  end
end
```

Listing 12: Memory evolution with causal tracking.

Evolution creates a *new* process (the child) rather than mutating the parent. This is consistent with immutable-data principles: the parent retains its original state, and the version history records the causal chain. The operation coordinates three subsystems: Memory (creating the child process), Version (recording the lineage entry in Mnesia), and Graph (establishing an `:evolved_into` edge).

5.6 Merge: Conflict Resolution

When two agents concurrently modify the same memory and their vector clocks are incomparable (indicating a conflict), the merge operation resolves it by creating a new memory that synthesizes both versions:

```elixir
@spec merge(pid(), pid(), map()) ::
  {:ok, pid()} | {:error, term()}
def merge(pid_a, pid_b, merged_data) do
```

```
    GenServer.call(pid_a, {:merge, pid_b, merged_data})
  end

  @impl true
  def handle_call({:merge, pid_b, merged_data},
                  _from, state_a) do
    case GenServer.call(pid_b, :get_state) do
      {:ok, state_b} ->
        case create(merged_data) do
          {:ok, merged_pid} ->
            {:ok, merged_state} =
              GenServer.call(merged_pid, :get_state)

            # Merge vector clocks: component-wise max
            merged_clock = merge_vector_clocks(
              state_a.vector_clock,
              state_b.vector_clock
            )
            GenServer.call(merged_pid,
              {:set_vector_clock, merged_clock})

            # Link both parents
            Graph.link(state_a.id, merged_state.id,
              :derived_from, %{operation: :merge})
            Graph.link(state_b.id, merged_state.id,
              :derived_from, %{operation: :merge})

            Version.record(state_a.id, merged_state.id,
              :correction, merged_clock)

            {:reply, {:ok, merged_pid}, state_a}
          {:error, reason} ->
            {:reply, {:error, reason}, state_a}
        end
      {:error, reason} ->
        {:reply, {:error, reason}, state_a}
    end
  end

  defp merge_vector_clocks(vc_a, vc_b) do
    Map.merge(vc_a, vc_b, fn _k, v1, v2 -> max(v1, v2) end)
  end
```

Listing 13: Merging concurrent versions with vector clock unification.

The merged clock takes the component-wise maximum of both parent clocks, guaranteeing that the merged version causally dominates both parents. Both parents are linked to the child via `:derived_from` edges, preserving the full conflict-resolution history.

5.7 Soft Deletes

Memories are never physically destroyed. The delete operation sets a `deleted_at` timestamp (the tombstone pattern):

```
@impl true
def handle_call(:delete, _from, state) do
  new_state = %{state | deleted_at: DateTime.utc_now()}
  persist(new_state)
  {:reply, :ok, new_state}
end

# Data access checks for tombstone
@impl true
def handle_call(:get_data, _from,
  %{deleted_at: d} = state) when not is_nil(d) do
  {:reply, {:error, :deleted}, state}
end

def handle_call(:get_data, _from, state) do
  touch_lru(state.id)
  {:reply, {:ok, state.data}, state}
end
```

Listing 14: Soft delete via tombstone timestamp.

The pattern match on `deleted_at` in the `:get_data` handler ensures that deleted memories return an error to callers, while the memory's graph edges and version history remain intact for lineage queries.

5.8 Persistence: Dual-Layer Write Path

Every state change is persisted to both ETS and Mnesia, with an LRU timestamp for eviction:

```
defp persist(state) do
  # ETS: working memory (microsecond access)
  :ets.insert(:memory_working, {state.id, state})

  # Mnesia: persistent knowledge (durable)
  Storage.save(state)

  # LRU tracking for eviction
  touch_lru(state.id)
  :ok
end

defp touch_lru(id) do
  :ets.insert(:memory_lru,
    {id, System.monotonic_time()})
end
```

Listing 15: Dual-layer persistence with LRU tracking.

The `System.monotonic_time/0` call provides a high-resolution, monotonically increasing timestamp suitable for LRU ordering. The `:memory_lru` ETS table is separate from `:memory_working` to avoid coupling access metadata with memory content.

6 Versioning and Causal Tracking

6.1 Change Reasons

Every version transition carries a *change reason* explaining why the memory changed. The four change reasons form a partial order with a subsumption relation:

```elixir
defmodule MemoryLayer.Version do
  @type change_reason ::
    :observation | :inference | :correction | :decay

  @spec subsumes?(change_reason(), change_reason()) ::
    boolean()
  def subsumes?(a, a), do: true
  def subsumes?(:correction, :observation), do: true
  def subsumes?(:inference, :observation), do: true
  def subsumes?(:decay, _), do: true
  def subsumes?(_, _), do: false

  @spec join_reasons(change_reason(), change_reason()) ::
    change_reason()
  def join_reasons(a, b) do
    cond do
      subsumes?(a, b) -> a
      subsumes?(b, a) -> b
      true -> :correction
    end
  end
end
```

Listing 16: Change reasons with subsumption ordering.

The semantics are:

- **`:observation`** — base level: new data was directly observed.
- **`:inference`** — extends observation: a conclusion derived from existing data.
- **`:correction`** — subsumes observation: a previous version was wrong.
- **`:decay`** — terminal: information is becoming stale or irrelevant.

The `join_reasons/2` function computes the least upper bound when merging version histories from concurrent updates, defaulting to `:correction` when neither reason subsumes the other.

6.2 Version Entries

Each version entry records the full causal context of a memory change:

```
@type version_entry :: %{
  memory_id: String.t(),
  parent_id: String.t() | nil,
  version: non_neg_integer(),
  reason: change_reason(),
  vector_clock: vector_clock(),
  timestamp: DateTime.t(),
  metadata: map()
}

@spec record(String.t() | nil, String.t(),
  change_reason(), vector_clock()) :: :ok
def record(parent_id, child_id, reason,
  parent_clock \\ %{}) do
  new_clock = increment_clock(parent_clock)
  version = next_version(parent_id)

  entry = %{
    memory_id: child_id,
    parent_id: parent_id,
    version: version,
    reason: reason,
    vector_clock: new_clock,
    timestamp: DateTime.utc_now(),
    metadata: %{}
  }

  {:atomic, :ok} =
    :mnesia.transaction(fn ->
      :mnesia.write({:versions, child_id, entry})
      :ok
    end)
  :ok
end
```

Listing 17: Version entry type and recording.

Version entries are stored in the Mnesia `:versions` table using the bag table type, which allows multiple entries per key. This enables the complete version history for a memory to be retrieved in a single query. The Mnesia transaction wrapper ensures atomicity: the version entry is either fully written or not at all.

6.3 History and Lineage Traversal

Two query functions provide different views of version history:

```
@spec history(String.t()) :: [version_entry()]
def history(memory_id) do
  {:atomic, entries} =
```

```
      :mnesia.transaction(fn ->
        :mnesia.match_object(
          {:versions, memory_id, :_})
      end)

    entries
    |> Enum.map(fn {:versions, _id, entry} -> entry end)
    |> Enum.sort_by(& &1.timestamp, DateTime)
  end

  @spec lineage(String.t()) :: [version_entry()]
  def lineage(memory_id) do
    case history(memory_id) do
      [] -> []
      [entry | _] = entries ->
        case entry.parent_id do
          nil -> entries
          parent_id -> lineage(parent_id) ++ entries
        end
    end
  end
```

Listing 18: Version history and lineage chain traversal.

The `history/1` function returns all version entries for a single memory, sorted by timestamp. The `lineage/1` function follows the `parent_id` chain recursively back to the root, building the complete causal path.

6.4 Vector Clocks

In multi-agent settings, memories may be updated concurrently by different agents. Vector clocks [Lamport, 1978, Fidge, 1988] establish causal ordering. Each agent maintains a counter; on write, the writing agent increments its component.

```
  @type vector_clock ::
    %{optional(atom() | String.t()) => non_neg_integer()}
  @type clock_ordering ::
    :before | :after | :concurrent | :equal

  @spec compare_clocks(vector_clock(), vector_clock()) ::
    clock_ordering()
  def compare_clocks(vc_a, vc_b) when vc_a == vc_b,
    do: :equal

  def compare_clocks(vc_a, vc_b) do
    all_keys = Map.keys(vc_a) ++ Map.keys(vc_b)
      |> Enum.uniq()

    comparisons = Enum.map(all_keys, fn key ->
      a_val = Map.get(vc_a, key, 0)
      b_val = Map.get(vc_b, key, 0)
      cond do
```

```
        a_val < b_val -> :less
        a_val > b_val -> :greater
        true -> :equal
      end
    end)

    has_less = :less in comparisons
    has_greater = :greater in comparisons

    cond do
      has_less and has_greater -> :concurrent
      has_less -> :before
      has_greater -> :after
      true -> :equal
    end
  end

  @spec increment_clock(vector_clock()) :: vector_clock()
  def increment_clock(vc) do
    Map.update(vc, node(), 1, &(&1 + 1))
  end

  @spec merge_clocks(vector_clock(), vector_clock()) ::
    vector_clock()
  def merge_clocks(vc_a, vc_b) do
    Map.merge(vc_a, vc_b,
      fn _key, v1, v2 -> max(v1, v2) end)
  end
```

Listing 19: Vector clock comparison for conflict detection.

Two clocks are *concurrent* when neither dominates the other (one has a strictly greater component for some agent while the other is strictly greater for a different agent). Concurrent clocks signal a conflict requiring explicit resolution through the merge operation (Section 5.6).

The vector clock operations are pure functions over immutable maps, making them safe for concurrent use and easy to reason about.

7 Graph Relationships

7.1 Edge Relations

The memory graph captures typed relationships between memory instances. Nine edge relations define the vocabulary of relationships:

```
defmodule MemoryLayer.Graph do
  use GenServer

  @type edge_relation ::
    :evolved_into  | :references   | :supersedes |
    :contradicts   | :supports     | :derived_from |
```

```
    :part_of        | :triggers      | :blocked_by

  @type edge :: %{
    from: String.t(),
    to: String.t(),
    relation: edge_relation(),
    metadata: map(),
    created_at: DateTime.t()
  }
```

Listing 20: The nine edge relation types.

Table 3 describes the semantics of each edge type:

Table 3: Edge relation semantics.

Relation	Semantics
`evolved_into`	Memory evolved into a new version (set by `evolve/3`)
`references`	Memory refers to another for context
`supersedes`	Memory replaces an older, outdated one
`contradicts`	Memory conflicts with another (triggers conflict resolution)
`supports`	Memory provides evidence for another
`derived_from`	Memory was derived from one or more parents (set by `merge/3`)
`part_of`	Memory is a component of a larger memory
`triggers`	Memory causes creation of another
`blocked_by`	Memory cannot proceed until another is resolved

7.2 Link and Unlink

Edges are stored in the Mnesia `:edges` table with a composite key `{from_id, to_id}`:

```
  @spec link(String.t(), String.t(), edge_relation(),
    map()) :: :ok | {:error, term()}
  def link(from_id, to_id, relation, metadata \\ %{}) do
    GenServer.call(__MODULE__,
      {:link, from_id, to_id, relation, metadata})
  end

  @impl true
  def handle_call({:link, from_id, to_id, relation,
    metadata}, _from, state) do
    edge = %{
      from: from_id, to: to_id,
      relation: relation, metadata: metadata,
      created_at: DateTime.utc_now()
    }
    result = :mnesia.transaction(fn ->
      :mnesia.write({:edges, {from_id, to_id}, edge})
    end)
    case result do
      {:atomic, _} -> {:reply, :ok, state}
```

```
      {:aborted, reason} -> {:reply, {:error, reason}, state}
    end
  end
```

Listing 21: Creating and soft-removing graph edges.

The `unlink/3` operation follows the same tombstone pattern as memory deletion: rather than physically removing the edge, it adds a `deleted_at` timestamp to the edge metadata. Query functions filter out tombstoned edges.

7.3 Graph Traversal

The graph module provides breadth-first traversal with configurable depth, direction, and relation filters:

```
  @type traverse_opts :: [
    max_depth: non_neg_integer(),
    direction: :forward | :reverse | :both,
    filter_relations: [edge_relation()]
  ]

  @spec traverse(String.t(), [edge_relation()],
    traverse_opts()) :: [String.t()]
  def traverse(start_id, pattern, opts \\ []) do
    GenServer.call(__MODULE__,
      {:traverse, start_id, pattern, opts})
  end

  # Private BFS implementation
  defp do_traverse(_, _, _, 0, visited), do: visited
  defp do_traverse([], _, _, _, visited), do: visited

  defp do_traverse(frontier, relations, direction,
    depth, visited) do
    new_visited = Enum.reduce(frontier, visited,
      &MapSet.put(&2, &1))

    next_frontier =
      Enum.flat_map(frontier, fn node_id ->
        edges = case direction do
          :forward -> do_edges_from(node_id)
          :reverse -> do_edges_to(node_id)
          :both ->
            do_edges_from(node_id) ++
            do_edges_to(node_id)
        end

        edges
        |> Enum.filter(& &1.relation in relations)
        |> Enum.map(fn edge ->
          case direction do
            :forward -> edge.to
```

```
            :reverse -> edge.from
            :both ->
              if edge.from == node_id,
                do: edge.to, else: edge.from
          end
        end)
        |> Enum.reject(&MapSet.member?(new_visited, &1))
      end)
      |> Enum.uniq()

    do_traverse(next_frontier, relations, direction,
      depth - 1, new_visited)
  end
```

Listing 22: BFS graph traversal with depth limiting.

The traversal uses a `MapSet` to track visited nodes, preventing cycles in the graph from causing infinite loops. The pattern parameter specifies which edge relations to follow, enabling targeted queries like "find all memories this one evolved into" or "find the full derivation chain".

7.4 Neighborhood Queries

The `neighborhood/2` function returns all memories connected to a given memory within a depth limit, along with their connecting edges:

```
  @spec neighborhood(String.t(), non_neg_integer()) ::
    [{String.t(), edge()}]
  def neighborhood(memory_id, max_depth \\ 2) do
    GenServer.call(__MODULE__,
      {:neighborhood, memory_id, max_depth})
  end
```

Listing 23: Neighborhood query: local subgraph extraction.

This is useful for building context around a memory: given a fact, the neighborhood reveals its supporting evidence, contradicting claims, derived decisions, and related tasks.

7.5 Conflict Detection

The graph module provides a convenience function for finding unresolved conflicts:

```
  @spec conflicts(String.t()) :: [{String.t(), String.t()}]
  def conflicts(memory_id) do
    edges_of_type(memory_id, :contradicts)
    |> Enum.map(fn edge -> {edge.from, edge.to} end)
  end
```

Listing 24: Detecting unresolved conflicts via contradicts edges.

When vector clocks detect concurrent modifications, the system creates `:contradicts` edges. Agents or operators can query for unresolved conflicts and apply the merge operation to resolve them.

8 Multi-Backend Storage Router

8.1 The StorageBackend Behaviour

The storage subsystem is built on an Elixir *behaviour*—the functional equivalent of an interface or protocol. Any module that implements these five callbacks can serve as a storage backend:

```
defmodule MemoryLayer.StorageBackend do
  @callback save(memory :: map()) ::
    :ok | {:error, term()}
  @callback recall(id :: String.t()) ::
    {:ok, map()} | {:error, :not_found}
  @callback search(query :: String.t(), opts :: keyword()) ::
    {:ok, [map()]} | {:error, term()}
  @callback delete(id :: String.t()) ::
    :ok | {:error, term()}
  @callback update(memory :: map()) ::
    :ok | {:error, term()}
end
```

Listing 25: StorageBackend behaviour: the five required callbacks.

This behaviour establishes a contract that all backends must fulfill. The Elixir compiler warns if a module declares `@behaviour MemoryLayer.StorageBackend` but fails to implement all callbacks.

8.2 The Storage Router

The `Storage` module implements the `StorageBackend` behaviour itself, acting as a router that coordinates ETS and Mnesia:

```
defmodule MemoryLayer.Storage do
  use GenServer
  @behaviour MemoryLayer.StorageBackend

  # Fan-out save: write to both backends
  @impl MemoryLayer.StorageBackend
  def save(memory) do
    # ETS: fast working memory
    :ets.insert(:memory_working, {memory.id, memory})

    # Mnesia: durable persistent storage
    result = :mnesia.transaction(fn ->
      :mnesia.write({:memories, memory.id, memory})
    end)

    case result do
      {:atomic, _} -> :ok
      {:aborted, reason} -> {:error, reason}
    end
  end
```

Listing 26: Storage router: fan-out save to ETS and Mnesia.

8.3 Tiered Recall with Automatic Promotion

The recall path checks ETS first (microsecond latency) and falls back to Mnesia (millisecond latency), automatically promoting Mnesia-only memories into ETS for subsequent fast access:

```
@impl MemoryLayer.StorageBackend
def recall(id) do
  case recall_from_ets(id) do
    {:ok, memory} ->
      {:ok, memory}
    {:error, :not_found} ->
      case recall_from_mnesia(id) do
        {:ok, memory} ->
          # Promote to ETS for fast future access
          :ets.insert(:memory_working, {id, memory})
          {:ok, memory}
        error ->
          error
      end
  end
end

defp recall_from_ets(id) do
  case :ets.lookup(:memory_working, id) do
    [{^id, memory}] -> {:ok, memory}
    [] -> {:error, :not_found}
  end
end

defp recall_from_mnesia(id) do
  result = :mnesia.transaction(fn ->
    :mnesia.read({:memories, id})
  end)
  case result do
    {:atomic, [{:memories, ^id, memory}]} ->
      {:ok, memory}
    {:atomic, []} ->
      {:error, :not_found}
    {:aborted, reason} ->
      {:error, reason}
  end
end
```

Listing 27: Tiered recall: ETS fast path with Mnesia fallback and promotion.

Note the use of the pin operator (`^id`) in the pattern matches to ensure the recalled memory's ID matches the requested one.

8.4 Cross-Backend Search

The search function dispatches to different backends based on the query type:

```
@impl MemoryLayer.StorageBackend
def search(query, opts \\ []) do
  backend = Keyword.get(opts, :backend, :all)
  limit = Keyword.get(opts, :limit, 50)
  include_deleted = Keyword.get(opts, :include_deleted, false)
  filter = Keyword.get(opts, :filter, %{})

  results = case backend do
    :ets      -> search_ets(query, filter)
    :exact    -> search_mnesia(query, filter)
    :mnesia   -> search_mnesia(query, filter)
    :semantic -> search_mnesia(query, filter)
    :graph    -> search_mnesia(query, filter)
    :all      -> merge_search_results([
                   search_ets(query, filter),
                   search_mnesia(query, filter)
                 ])
  end

  filtered = results
    |> maybe_exclude_deleted(include_deleted)
    |> Enum.take(limit)

  {:ok, filtered}
end
```

Listing 28: Multi-backend search dispatch.

The `:semantic` and `:graph` backends currently fall back to Mnesia search; they are extension points for future backends (ChromaDB for vector search, FalkorDB for graph traversal). When dispatching to `:all`, results from both backends are merged with deduplication by memory ID.

8.5 Text Matching and Filtering

The search pipeline applies structural filtering (matching map fields) and text matching (substring search across the serialized memory):

```
defp apply_filter(memories, filter) when filter == %{},
  do: memories

defp apply_filter(memories, filter) do
  Enum.filter(memories, fn memory ->
    Enum.all?(filter, fn {key, value} ->
      Map.get(memory, key) == value
    end)
  end)
end

```

```
  defp apply_text_match(memories, ""), do: memories

  defp apply_text_match(memories, query) do
    query_lower = String.downcase(query)
    Enum.filter(memories, fn memory ->
      memory
      |> inspect()
      |> String.downcase()
      |> String.contains?(query_lower)
    end)
  end
```

Listing 29: Filter and text-match pipeline.

The filter pipeline demonstrates Elixir's pattern matching on function heads: the empty-filter and empty-query cases are handled by dedicated function clauses that short-circuit the processing.

8.6 LRU Eviction

Working memory (ETS) is bounded. The eviction function demotes least-recently-used memories to Mnesia-only storage:

```
  @spec evict_lru(non_neg_integer()) :: non_neg_integer()
  def evict_lru(keep \\ 1000) do
    current_size = :ets.info(:memory_working, :size)

    if current_size > keep do
      lru_entries = :ets.tab2list(:memory_lru)
        |> Enum.sort_by(fn {_id, time} -> time end)

      to_evict = Enum.take(lru_entries,
        current_size - keep)

      Enum.each(to_evict, fn {id, _time} ->
        :ets.delete(:memory_working, id)
        :ets.delete(:memory_lru, id)
      end)

      length(to_evict)
    else
      0
    end
  end
```

Listing 30: LRU eviction from working memory.

Evicted memories are not lost—they remain in Mnesia and will be promoted back to ETS on the next recall. This creates a natural working-set behavior: frequently accessed memories stay in fast storage, while rarely accessed memories settle into durable-only storage.

8.7 Working Memory vs. Persistent Knowledge

The dual-layer architecture mirrors human cognitive architecture:

Table 4: Storage tier comparison.

Property	ETS (Working Memory)	Mnesia (Persistent Knowledge)
Latency	Microseconds	Milliseconds
Durability	Volatile (lost on crash)	Durable (disk-backed)
Capacity	Bounded (LRU eviction)	Unbounded
Access	In-process, concurrent reads	Transactional, distributed
Analogy	Working memory [Baddeley, 1992]	Long-term memory

The promotion/demotion protocol:

1. **Create**: written to both ETS and Mnesia (fan-out).
2. **Recall**: if in ETS, return immediately; if only in Mnesia, load into ETS (promotion).
3. **Evict**: remove from ETS, retain in Mnesia (demotion).
4. **Bound**: LRU eviction triggers when ETS size exceeds threshold.

9 Reliability Guarantees

9.1 ACID Properties via Mnesia Transactions

All Mnesia operations in the memory layer are wrapped in transactions:

1. **Atomicity**: Mnesia transactions are all-or-nothing; a failed write rolls back all changes within the transaction.
2. **Consistency**: schema validation via struct enforcement and typespecs ensures that only valid memory instances are persisted.
3. **Isolation**: Mnesia's transaction manager serializes concurrent writes to the same memory.
4. **Durability**: Mnesia's `disc_copies` table type writes to both RAM and disk, surviving node restarts.

9.2 Type Preservation Under Round-Trip

Theorem 9.1 (Round-Trip Type Preservation)**.** *For any schema type S and memory instance m of type S: if m is saved to storage and subsequently recalled, the recalled data is type-equivalent to m. That is,* `recall(save(m)).data` *reconstructs a struct of the same type with identical field values.*

Proof sketch. The save path serializes the Elixir term (including struct metadata) via `:erlang.term_to_binary/1` when stored in Mnesia, or stores the term directly in ETS. The recall path retrieves the exact term. Since Elixir structs carry their module name as part of the term representation, and ETS/Mnesia preserve Erlang terms exactly, the recalled value is identical to the stored value. The `schema_name` field provides a secondary check: the Schema Registry resolves it to the correct module, confirming type consistency. □

9.3 Deduplication Soundness

Proposition 9.2 (Deduplication Soundness)**.** *Content hashing is sound for deduplication: if $h(m_1) = h(m_2)$, then $m_1.\texttt{data} = m_2.\texttt{data}$ with overwhelming probability ($1 - 2^{-256}$).*

The SHA-256 hash is computed over the canonical binary serialization of the data struct. The `:erlang.term_to_binary/1` function produces a deterministic encoding for any Elixir term, ensuring that structurally identical data always produces the same hash.

9.4 Conflict Detection Completeness

Proposition 9.3 (Conflict Detection)**.** *Vector clock comparison detects all concurrent modifications. If two agents independently modify a memory without observing each other's changes, their resulting vector clocks will be incomparable, and `compare_clocks/2` will return `:concurrent`.*

This follows directly from the properties of vector clocks [Fidge, 1988]: the comparison returns `:concurrent` exactly when neither clock component-wise dominates the other, which occurs precisely when the modifications are causally independent.

9.5 Soft Delete Lineage Preservation

Proposition 9.4 (Lineage Preservation)**.** *Soft deletion preserves the complete graph structure. All edges incident to a deleted memory remain in the Mnesia `:edges` table, and lineage queries (`Version.lineage/1`, `Graph.traverse/3`) can still traverse through deleted nodes.*

This is essential for maintaining an accurate history: even when a fact is retracted (soft-deleted), the chain of reasoning that led to it—and the corrections that followed—remains queryable.

10 MCP Server Interface

ContextFS exposes memory operations via the Model Context Protocol (MCP) [Anthropic, 2024], enabling any MCP-compatible client (Claude, GPT, custom agents) to access the memory layer. The MCP server translates tool calls into GenServer messages, ensuring that all access goes through the typed memory interface.

Table 5: MCP tools exposed by ContextFS.

Tool	Description
`contextfs_save`	Create and persist a new typed memory
`contextfs_recall`	Retrieve a memory by ID with full type reconstruction
`contextfs_search`	Multi-backend search (exact, semantic, graph)
`contextfs_evolve`	Create a new version linked to parent with change reason
`contextfs_link`	Establish a typed edge between two memories
`contextfs_delete`	Soft-delete (tombstone) a memory
`contextfs_update`	Update fields of an existing memory
`contextfs_list`	List memories by type, tag, or time range
`contextfs_index`	Trigger re-indexing of embedding vectors
`contextfs_sync`	Synchronize local and cloud backends

An example MCP tool call:

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "contextfs_save",
    "arguments": {
      "content": "Login test requires 60s timeout",
      "memory_type": "fact",
      "metadata": {
        "confidence": 0.95,
        "source": "test_run_2026_03_10"
      },
      "tags": ["testing", "timeout", "login"]
    }
  },
  "id": 42
}
```

Listing 31: MCP tool call for saving a typed memory.

The MCP server receives this JSON-RPC message, constructs a `FactData` struct from the arguments, calls `Memory.create/1`, and returns the resulting memory ID and metadata to the client.

11 Case Studies

11.1 Agent Learning from Past Executions

Consider a testing agent that executes web application tests across multiple sessions. Without persistent memory, each session starts with no knowledge of past failures. With the memory layer:

1. After each test run, the agent creates `episodic` memories recording what was tested, what failed, and what succeeded.

2. Failed test patterns are stored as `fact` memories with confidence scores and suspected causes.
3. When the agent encounters a similar failure in a subsequent session, search retrieves relevant past episodes, and the agent applies learned strategies.
4. Over time, `procedural` memories accumulate: "When test X fails with error Y, the root cause is usually Z."

The version lineage tracks how the agent's understanding evolves:

```
# Session N: Agent encounters a test failure
{:ok, episode} = Memory.create(%EpisodicData{
  event: "login_test_timeout",
  context: %{url: "https://app.example.com/login"},
  outcome: :failure,
  duration_ms: 30_000
})

# Search for similar past failures
{:ok, similar} = Storage.search(
  "login test timeout failure",
  backend: :all, limit: 5
)

# Found a previous procedural fix -- apply and record
{:ok, _evolved} = Memory.evolve(episode, %{
  outcome: :success,
  context: %{
    url: "https://app.example.com/login",
    resolution: "Increased timeout to 60s per PROC-442"
  }
}, :correction)
```

Listing 32: Agent learning: recording and evolving from test failures.

11.2 Cross-Session Continuity

A software engineering agent works on a multi-day project. Between sessions, the user may make changes that invalidate the agent's assumptions:

```
# Session start: recall last session state
{:ok, sessions} = Storage.search("",
  backend: :exact,
  filter: %{schema_name: "session"},
  limit: 1
)
last_session = hd(sessions)

# Recall all in-progress TODOs
{:ok, todos} = Storage.search("",
  backend: :exact,
```

```
  filter: %{schema_name: "todo"}
)

# Check for invalidated assumptions
for todo <- todos do
  deps = Graph.traverse(todo.id, [:references],
    max_depth: 2)
  for dep_id <- deps do
    {:ok, dep} = Storage.recall(dep_id)
    if stale?(dep) do
      # Evolve the stale fact with correction reason
      Memory.evolve(dep.id, %{
        confidence: 0.3
      }, :decay)
    end
  end
end
```

Listing 33: Cross-session context restoration and assumption checking.

The graph lineage preserves the full narrative: original plan, partial execution, external change, plan revision.

11.3 Multi-Agent Shared Knowledge Base

Multiple agents collaborate on a shared codebase, each responsible for different modules:

1. Agent A (frontend) discovers a bug and creates an `issue` memory.
2. Agent B (backend) receives the issue via Mnesia replication.
3. Agent B investigates and creates a `decision` memory linked to the issue via `:derived_from`.
4. Agent A detects the decision via graph traversal and creates `task` memories linked via `:triggers`.

Vector clocks detect concurrent edits:

```
# Agent A writes with its clock component
vc_a = %{agent_a: 5, agent_b: 3}

# Agent B writes concurrently
vc_b = %{agent_a: 4, agent_b: 4}

# Neither dominates: conflict detected
case Version.compare_clocks(vc_a, vc_b) do
  :concurrent ->
    # Flag for merge resolution
    Graph.link(mem_a_id, mem_b_id, :contradicts,
      %{resolution: :pending})

    # Resolve by merging
    {:ok, _merged} = Memory.merge(
```

```
      mem_a_pid, mem_b_pid,
      reconciled_data
    )
  :before -> :ok
  :after  -> :ok
end
```

Listing 34: Multi-agent conflict detection via vector clocks.

12 Discussion

12.1 Comparison to Unix VFS

The Unix Virtual Filesystem Switch and our Storage router share the goal of providing a uniform interface over heterogeneous storage backends. However, the VFS operates at the byte-stream level: `read(fd, buf, count)` makes no type promises about the bytes returned. Our Storage router operates at the typed-schema level: `recall(id)` returns a memory with compile-time and runtime type guarantees.

The VFS uses inodes as the unit of storage; we use typed memory instances. The VFS organizes files in a tree hierarchy; we organize memories in a labeled graph. The VFS supports a fixed set of operations; our `StorageBackend` behaviour supports extensible operation sets.

12.2 Comparison to Plan 9

Plan 9's insight was that *everything should be a file*. Our analogous principle is that *everything an agent knows should be a typed memory*—including execution traces (`agent_run`), task queues (`todo`), and configuration (`config`).

Plan 9 used synthetic file servers to present dynamic state as files. We use MCP tools to present dynamic memory state as structured, queryable knowledge. The key advance is that our "files" carry schemas, versions, and graph relationships.

12.3 Comparison to Knowledge Graphs

Knowledge graphs (Wikidata, Neo4j) store entities and relationships but typically lack:

- **Schema-indexed typing**: KG nodes are loosely typed with optional properties. Our structs enforce strict schemas with compile-time and runtime checking.
- **Causal versioning**: KGs store current state; versioning is external. Our Version module integrates versioning as a first-class concern with causal annotations.
- **Multi-modal storage**: KGs and vector stores are typically separate systems. Our Storage router unifies them behind a single behaviour.

12.4 Relation to Cognitive Architectures

Soar [Laird, 2012] distinguished working memory, long-term declarative, procedural, and episodic memory—a taxonomy our 24 types refine and extend. ACT-R [Anderson et al.,

2004] formalized memory retrieval as activation-based competition, which inspired our LRU-based promotion/demotion between ETS and Mnesia. Generative agents [Park et al., 2023] demonstrated that LLM agents benefit from structured memory stores, though their implementation lacked type safety and formal versioning.

12.5 Connections to Category Theory

For readers familiar with category theory, the design has natural categorical interpretations:

- The **Schema module** defines a category whose objects are the 24 typed schemas and whose morphisms are schema transformations (field projections, type coercions). The `BaseSchema` is an initial object.
- The **Storage router** is a coproduct of storage backends: it exposes the union of all backend capabilities through injection morphisms.
- The **Memory GenServer** acts as a functor from the schema category to a storage category, mapping each schema type to its storage representation while preserving type structure.
- The **evolve/merge/split operations** form a monad on the category of memory instances: evolve is the unit (embedding a memory into a "pending change" context), merge is the multiplication (flattening nested changes), and the monad laws correspond to associativity and identity of operation composition.
- **Version tracking** is a functor from a timeline category (a thin category where objects are timestamps and morphisms are temporal orderings) to the storage category.
- The **graph relationships** exhibit 2-categorical structure: edges are 1-morphisms between memories, and equivalences between different edge paths (e.g., "evolved_into" vs. "supersedes") are 2-morphisms.

These connections are not decorative—they explain *why* the design composes cleanly. The functor laws guarantee that type structure is preserved across storage round-trips. The monad laws guarantee that chained evolutions and merges produce consistent results regardless of grouping. The coproduct construction guarantees that adding a new storage backend preserves all existing capabilities.

12.6 Relation to Version Control

Git [Chacon and Straub, 2014] uses content-addressable storage with a DAG of commits. Our version lineage is analogous but operates on typed memory instances rather than file trees. Our change reasons provide semantic annotations that Git commit messages cannot enforce structurally. Our vector clocks provide causal ordering in distributed settings, whereas Git relies on a total ordering of commits within each branch.

12.7 Limitations and Future Work

Several limitations merit future investigation:

1. **Garbage collection**: the tombstone pattern means storage grows monotonically. A formal garbage collection policy (analogous to generational GC) would bound storage while preserving essential lineage.

2. **Schema migration**: when a schema's definition changes (fields added or removed), existing memories must be migrated. The framework handles this through schema transformations, but the migration strategy needs automation.

3. **Scalability**: Mnesia's distribution model has known limitations beyond a few hundred nodes. For planetary-scale deployments, a CockroachDB or FoundationDB backend would be needed.

4. **Embedding drift**: as embedding models are updated, previously computed vectors may become inconsistent. Re-indexing strategies need formal treatment.

5. **Privacy and forgetting**: the "right to be forgotten" conflicts with the tombstone pattern. A cryptographic erasure scheme would reconcile these requirements.

6. **Vector search backend**: the current implementation stubs out semantic search. A ChromaDB or Qdrant backend implementing the `StorageBackend` behaviour would complete the multi-modal search capability.

7. **Graph database backend**: similarly, a FalkorDB or Neo4j backend would enable more efficient graph traversal than the current Mnesia-based BFS.

13 Related Work

Agent architectures with memory. Soar [Laird, 2012] and ACT-R [Anderson et al., 2004] established the cognitive architecture paradigm with distinct memory systems. MemGPT [Packer et al., 2023] introduced OS-inspired memory tiers for LLMs, with a main context (working memory) and external storage (archival memory). Our framework provides stronger type guarantees and richer relationship structure than MemGPT. Generative agents [Park et al., 2023] demonstrated the value of structured memory for believable agent behavior but used ad-hoc storage without formal typing.

Type-theoretic databases. Categorical databases (CQL) [Spivak, 2014, Schultz et al., 2017] formalize database schemas as categories and instances as functors. Our design adapts these ideas to agent memory with the additions of causal versioning, graph lineage, and multi-backend routing.

Distributed consistency. Dynamo [DeCandia et al., 2007] popularized vector clocks for eventually consistent storage. CRDTs [Shapiro et al., 2011] provide conflict-free replicated data types. Our adaptation applies vector clocks to agent memory with the constraint that conflicts require typed merge resolution rather than automatic convergence.

Memory in LLM systems. LangChain [LangChain, 2023] provides buffer, summary, and entity memory modules. Our framework subsumes these with formal type guarantees, graph relationships, and causal versioning.

BEAM/OTP for knowledge systems. The use of ETS and Mnesia for knowledge storage has precedent in telecommunications systems [Mattsson et al., 1999, Virding et al., 1996], where Mnesia was designed for soft-real-time, fault-tolerant data management. Our contribution is applying these tools to the agent cognition domain with typed schemas and causal versioning.

14 Conclusion

We have presented the design and implementation of a typed memory layer for AI agent cognition, realized in Elixir/OTP. The key contributions are:

1. A **type system** with 24 schema-indexed memory types organized into six subcategories, enforced at both compile time (Dialyzer) and runtime (struct enforcement), with a GenServer-based registry for dynamic type resolution.

2. A **process-per-memory architecture** using OTP GenServers, providing natural isolation, concurrent access, and fault-tolerant supervision for individual memory instances.

3. A **versioning framework** with four causal change reasons (observation, inference, correction, decay), parent-child lineage chains stored in Mnesia transactions, and vector clocks for multi-agent conflict detection.

4. A **graph relationship manager** with nine typed edge relations, BFS traversal with depth limiting, neighborhood queries, and conflict detection.

5. A **multi-backend storage router** implementing a behaviour protocol, with ETS for microsecond working memory, Mnesia for durable persistent knowledge, tiered recall with automatic promotion, and LRU eviction.

6. A **production deployment** as ContextFS with an MCP server for universal client access.

7. **Case studies** demonstrating agent learning, cross-session continuity, and multi-agent collaboration with conflict resolution.

The memory layer is the third component of the AI Operating System, building on the kernel (Part I) and scheduler (Part II). Together, these three components provide process management, resource allocation, and persistent knowledge—the foundations upon which higher-level agent capabilities can be constructed. Part IV will address the tool interface layer, formalizing how agents interact with external systems through typed, composable tool protocols.

The functional programming approach is not mere aesthetic preference: immutable data eliminates shared-state concurrency bugs, pattern matching enables exhaustive type dispatch, behaviours provide extensible contracts, and OTP supervision trees deliver fault tolerance. The Elixir implementation demonstrates that these principled abstractions yield both formal guarantees and practical performance.

Agent cognition requires memory that is typed, versioned, searchable, and graph-structured. The filesystem analogy is apt but insufficient: what agents need is not a filesystem but a *knowledge system*. The typed memory layer provides exactly this.

References

J. R. Anderson, D. Bothell, M. D. Byrne, S. Douglass, C. Lebiere, and Y. Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.

Anthropic. Model Context Protocol specification. Technical report, Anthropic, 2024. https://modelcontextprotocol.io.

J. Armstrong. *Programming Erlang: Software for a Concurrent World*. Pragmatic Bookshelf, 2nd edition, 2013.

A. Baddeley. Working memory. *Science*, 255(5044):556–559, 1992.

S. Chacon and B. Straub. *Pro Git*. Apress, 2nd edition, 2014.

G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels. Dynamo: Amazon's highly available key-value store. In *Proc. 21st ACM SOSP*, pages 205–220, 2007.

P. J. Denning. Virtual memory. *ACM Computing Surveys*, 2(3):153–189, 1970.

J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proc. NAACL-HLT*, pages 4171–4186, 2019.

C. J. Fidge. Timestamps in message-passing systems that preserve the partial ordering. In *Proc. 11th Australian Computer Science Conference*, pages 56–66, 1988.

D. K. Gifford, P. Jouvelot, M. A. Sheldon, and J. W. O'Toole Jr. Semantic file systems. In *Proc. 13th ACM SOSP*, pages 16–25, 1991.

A. Hogan, E. Blomqvist, M. Cochez, et al. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.

M. Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

J. E. Laird. *The Soar Cognitive Architecture*. MIT Press, 2012.

L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

LangChain. LangChain documentation: Memory. https://docs.langchain.com/docs/components/memory, 2023.

P. Lewis, E. Perez, A. Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proc. NeurIPS*, pages 9459–9474, 2020.

R. Love. *Linux Kernel Development*. Addison-Wesley, 3rd edition, 2010.

H. Mattsson, H. Nilsson, and C. Wikström. Mnesia — A distributed robust DBMS for telecommunications applications. In *Proc. PADL*, pages 152–163, 1999.

M. Minsky. A framework for representing knowledge. MIT AI Laboratory Memo 306, 1974.

C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proc. 36th ACM UIST*, pages 1–22, 2023.

R. Pike, D. Presotto, S. Dorward, et al. Plan 9 from Bell Labs. *Computing Systems*, 8(3):221–254, 1995.

M. R. Quillian. Semantic memory. In M. Minsky, editor, *Semantic Information Processing*, pages 227–270. MIT Press, 1968.

N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proc. EMNLP-IJCNLP*, pages 3982–3992, 2019.

D. M. Ritchie and K. Thompson. The UNIX time-sharing system. *Communications of the ACM*, 17(7):365–375, 1974.

P. Schultz, D. I. Spivak, and C. Vasilakopoulou. Algebraic databases. *Theory and Applications of Categories*, 32(16):547–619, 2017.

M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. Conflict-free replicated data types. In *Proc. SSS*, pages 386–400, 2011.

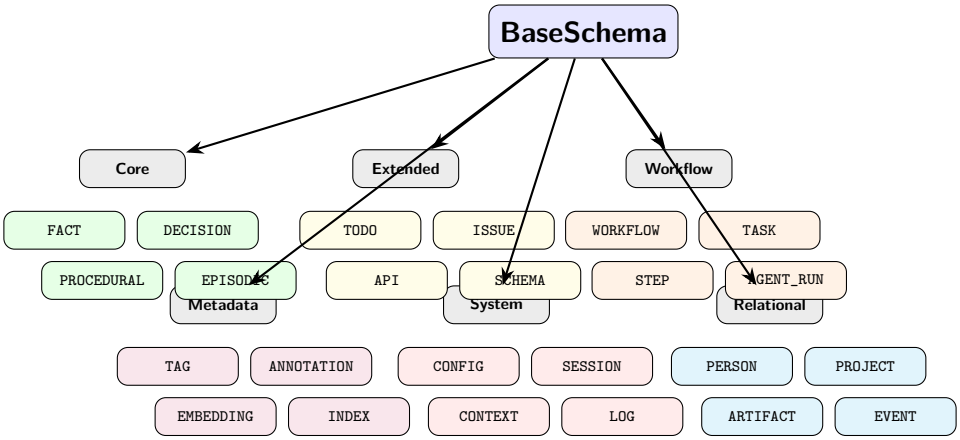
D. I. Spivak. *Category Theory for the Sciences*. MIT Press, 2014.

D. Thomas and A. Hunt. *Programming Elixir ≥ 1.6*. Pragmatic Bookshelf, 2018.

R. Virding, C. Wikström, and M. Williams. *Concurrent Programming in Erlang*. Prentice Hall, 2nd edition, 1996.

A Complete Memory Type Hierarchy

The following diagram shows the 24 memory types organized by subcategory, all deriving from `BaseSchema`:



B OTP Application Startup

The `MemoryLayer` application module initializes ETS tables and Mnesia before starting the supervision tree:

```
defmodule MemoryLayer do
  use Application

  @impl true
  def start(_type, _args) do
    # Create ETS tables for working memory
    :ets.new(:memory_working, [
      :set, :public, :named_table,
      read_concurrency: true
    ])
    :ets.new(:memory_lru, [
      :set, :public, :named_table
    ])

    # Initialize Mnesia schema and tables
    :mnesia.create_schema([node()])
    :mnesia.start()
    :mnesia.create_table(:memories, [
      attributes: [:id, :data],
      type: :set,
      disc_copies: [node()]
    ])
    :mnesia.create_table(:versions, [
      attributes: [:id, :entry],
      type: :bag,
      disc_copies: [node()]
    ])
    :mnesia.create_table(:edges, [
      attributes: [:key, :edge],
      type: :bag,
      disc_copies: [node()]
    ])

    # Start supervision tree
    children = [
      MemoryLayer.Schema.Registry,
      MemoryLayer.Graph,
      MemoryLayer.Storage
    ]

    opts = [
      strategy: :one_for_one,
      name: MemoryLayer.Supervisor
    ]
    Supervisor.start_link(children, opts)
  end
end
```

Listing 35: Application startup: ETS and Mnesia initialization.

The `:one_for_one` supervision strategy restarts individual processes independently: if the Graph GenServer crashes, the Schema Registry and Storage continue running unaffected.

C Mix Project Configuration

The project dependencies are minimal, leveraging OTP's built-in capabilities:

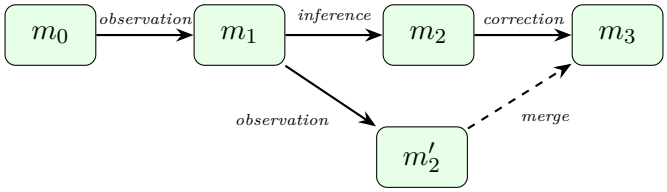
```
defmodule MemoryLayer.MixProject do
  use Mix.Project

  def project do
    [
      app: :memory_layer,
      version: "0.1.0",
      elixir: "~> 1.16",
      start_permanent: Mix.env() == :prod,
      deps: deps()
    ]
  end

  def application do
    [
      extra_applications: [:logger, :mnesia, :crypto],
      mod: {MemoryLayer, []}
    ]
  end

  defp deps do
    [
      {:uuid, "~> 1.1"},
      {:jason, "~> 1.4"},
      {:ex_doc, "~> 0.31", only: :dev, runtime: false},
      {:dialyxir, "~> 1.4", only: [:dev, :test],
        runtime: false}
    ]
  end
end
```

Listing 36: mix.exs: project configuration.

The only runtime dependencies beyond OTP are `uuid` (for generating memory identifiers) and `jason` (for JSON serialization in the MCP interface). ETS, Mnesia, and `:crypto` are all built-in OTP applications.

D Version Chain Diagram

A typical memory evolution chain with change reasons:



In this example, m_0 is created via observation, then m_1 is inferred from it. A concurrent modification produces m_2 and m_2'. The conflict is resolved by merging into m_3 with a correction reason.